

Memory Efficient Doubly Linked List – Insert At END –Space Complexity and Algorithm

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr){ break; }
        prev = curr;
        curr = next;
    }
    curr->npx = XOR(prev, newNode);
    newNode->npx = XOR(curr, nullptr);
    cout << data << " inserted at the end." << endl;
}

.....

case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Space Complexity Analysis: XOR Linked List (Insert At End)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameter:** *int data.*
 - **Analysis:** *The data parameter is a single integer. It takes a constant amount of space regardless of the list size.*
 - **Complexity:** **$O(1)$**
- **Full Data Definition:**
 - **Function Data:** *The int data parameter + the existing XOR list.*
 - **Analysis:** *To find the end of the list, the function must access every node in the sequence. If there are n nodes, the input data structure being operated on is proportional to n .*
 - **Complexity:** **$O(n)$**

2. Auxiliary Space Complexity

Definition: *This measures the extra /temporary space used during the execution of the function.*

- **Node Allocation:** *`Node *newNode = (Node *)malloc(sizeof(Node));`. This creates exactly **one** new node.*
- **Local Pointer Variables:** *The function declares three local pointers: `curr`, `prev`, and `next`.*
 - *Even though the while loop runs n times, these variables are **reused** in each iteration. They do not multiply as the list grows.*
- **Traversal:** *The traversal is iterative, not recursive. This means no extra memory is added to the call stack as the list size increases.*
- **Operations:** *The XOR calculation is a bitwise operation that happens in CPU registers.*

Conclusion: *The amount of extra memory used by this function is fixed. It doesn't matter if the list has 5 nodes or 5,000 nodes; the function only uses space for one new node and three pointers.*

- **Auxiliary Space Complexity:** **$O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter – only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full – data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

*In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity** ($O(1)$) is the expected answer.*

Algorithm: Insert at End (XOR Doubly Linked List)

XORINSEND(START, ITEM):

*((Node *)malloc(sizeof(Node)) represents the availability of space for the data; let's name it AVAIL.))*

1. **[ALLOCATE NEW NODE]**
Set NEWPTR := AVAIL
2. **[MEMORY ALLOCATED?]**
If NEWPTR = NULL, then:
Write: "Error: Memory allocation failed."
Return and EXIT.
[End of If structure]
3. **[COPY DATA]**
Set INFO[NEWPTR] := ITEM
4. **[CHECK IF LIST IS EMPTY]**
If START = NULL, then:
 - a. *Set NEXTXORPTR[NEWPTR] := NULL $\oplus$ NULL*
 - b. *Set START := NEWPTR*
 - c. *Write: ITEM, " inserted at the end."*
 - d. *Return and EXIT.***[End of If structure]**
5. **[INITIALIZE TRAVERSAL]**
Set CURRPTR := HEAD
Set PREVPTR := NULL
6. **[TRAVERSE TO THE LAST NODE]**
Repeat while CURRPTR $\neq$ NULL:
 - a. *Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]*
 - b. *If NEXTTEMPPTR = NULL, then:*
Break and Exit from the loop
 - c. *Set PREVPTR := CURRPTR*
 - d. *Set CURRPTR := NEXTTEMPPTR***[End of Loop]**

7. [UPDATE POINTERS FOR INSERTION]

a. Set NEXTXORPTR[CURRPTR] := PREVPTR $\oplus$ NEWPTR $\left(\begin{array}{l} \text{Update} \\ \text{old last node's NPX} \\ \text{to point to the new node} \end{array} \right)$

b. Set NEXTXORPTR [NEWPTR] := CURRPTR $\oplus$ NULL $\left(\begin{array}{l} \text{New node's NPX} \\ \text{points back} \\ \text{to old last node} \\ \text{and forward to NULL} \end{array} \right)$

8. [PRINT MESSAGE]

Write: ITEM, " inserted at the end. "

9. EXIT
